

Aether — Metal3 Bare Metal

Bare Metal Provisioning via Kubernetes

Provision physical servers from the same workload spec used for containers and VMs. Aether generates BareMetalHost CRDs with annotation-driven hardware configuration.

BareMetalHost CRD • IPMI/BMC • UEFI • Hardware Matching

What is Metal3?

Metal3 (Metal-Kubed) brings bare-metal server management into the Kubernetes ecosystem using the BareMetalHost CRD and IroniC for hardware provisioning.



Bare Metal as a Resource

Metal3 treats physical servers as Kubernetes resources. The BareMetalHost CRD represents each machine with its BMC credentials, hardware profile, and provisioning state.



IroniC-Powered Provisioning

Under the hood, Metal3 uses OpenStack IroniC for PXE boot, disk imaging, and IPMI/Redfish power management. Aether abstracts all of this behind annotations on your workload spec.



BMC Integration

IPMI, Redfish, and iDRAC protocols are supported for power control, boot order management, and serial console access. Credentials are stored as Kubernetes Secrets.



Fleet Management

Manage hundreds of bare-metal hosts from a single Kubernetes cluster. Metal3 handles hardware inspection, provisioning, and deprovisioning at scale.

Metal3 turns your data center into a Kubernetes-native resource pool. Aether makes provisioning a bare-metal server as simple as deploying a container.

BareMetalHost CRD Generation

Aether's Metal3 adapter reads annotations from your workload spec to generate a fully configured BareMetalHost resource.

Workload Spec with Annotations

```
apiVersion: aether/v1
kind: Workload
metadata:
  name: gpu-node-01
  owner: infra-team
  project: ml-cluster
  annotations:
    aether.io/boot-mac-address: "00:1A:2B:3C:4D:5E"
    aether.io/image-url: "https://images.example.com/coreos.qcow2"
    aether.io/image-checksum-url: "https://images.example.com/coreos.qcow2.md5"
    aether.io/boot-mode: "UEFI"
spec:
  requirements:
    cpu: "64"
    memory: 256Gi
    storage: 2Ti
    gpu:
      count: 8
      vendor: nvidia
  runtime:
    preferred: metal
    allow: [metal]
```

Generated BareMetalHost

```
apiVersion: metal3.io/v1alpha1
kind: BareMetalHost
metadata:
  name: gpu-node-01
  namespace: metal3-system
  labels:
    app: gpu-node-01
    managed-by: aether
  annotations:
    aether.io/cpu-cores: "64"
    aether.io/memory-mb: "262144"
    aether.io/gpu-count: "8"
spec:
  online: true
  bootMACAddress: "00:1A:2B:3C:4D:5E"
  bootMode: UEFI
  image:
    url: "https://images.example.com/coreos.qcow2"
    checksum: "https://images.example.com/coreos.qcow2.md5"
  userData:
    name: gpu-node-01-userdata
    namespace: metal3-system
  networkData:
    name: gpu-node-01-networkdata
    namespace: metal3-system
  rootDeviceHints:
    deviceName: /dev/sda
    minSizeGigabytes: 2048
    hardwareProfile: unknown
```

Hardware Matching

Aether stores hardware requirements as annotations on the BareMetalHost, enabling intelligent matching against available physical infrastructure.



CPU Matching

The `aether.io/cpu-cores` annotation stores the parsed CPU core count. Aether converts Kubernetes-style CPU values (e.g., "64" or "64000m") into integer core counts for matching.



Memory Matching

The `aether.io/memory-mb` annotation stores memory in megabytes, converted from the spec's Gi/Mi notation. This enables precise matching against physical DIMM configurations.



GPU Matching

The `aether.io/gpu-count` and `aether.io/gpu-vendor` annotations match against available GPU hardware. Supports NVIDIA, AMD, and Intel discrete GPUs.



Storage Matching

Root device hints specify minimum disk size via `rootDeviceHints.minSizeGigabytes`. Aether converts the spec's storage field to gigabytes for device selection.

Provisioning Lifecycle

Registering



Inspecting



Available



Provisioning



Provisioned

- 1 Inspecting**
Metal3 powers on the host via BMC and collects hardware inventory: CPU model, DIMM count, NIC MACs, disk serial numbers, and GPU devices.
- 2 Provisioning**
The host PXE boots from Ironic, writes the OS image to disk, applies `userData` and `networkData`, and reboots into the installed operating system.
- 3 Provisioned**
The host is running and ready. Aether reports it as Running with health checks monitoring the BMC and SSH connectivity.

Annotation Reference

Complete reference for all Aether annotations used in Metal3 bare-metal provisioning.

Annotation	Required	Description	Example
aether.io/boot-mac-address	✓	MAC address for PXE boot NIC	00:1A:2B:3C:4D:5E
aether.io/image-url	✓	URL of the OS disk image (qcow2, raw, iso)	https://images.example.com/coreos.qcow2
aether.io/image-checksum-url	✓	URL of the image checksum file	https://images.example.com/coreos.md5
aether.io/boot-mode		Boot mode: UEFI (default) or legacy	UEFI
aether.io/cpu-cores		Auto-generated from requirements.cpu	64
aether.io/memory-mb		Auto-generated from requirements.memory	262144
aether.io/gpu-count		Auto-generated from requirements.gpu.count	8
aether.io/gpu-vendor		Auto-generated from requirements.gpu.vendor	nvidia

Default Namespace: metal3-system

The Metal3 adapter defaults to the `metal3-system` namespace, which is where the Metal3 operator and Ironi components typically run. Override with `--namespace` or `AETHER_NAMESPACE` as needed.

Annotations are the bridge between Aether's universal workload spec and Metal3's hardware-specific configuration. Required annotations define the physical target; auto-generated annotations enable intelligent fleet matching.

Provision Bare Metal

From YAML to running hardware in minutes. Aether brings the same declarative workflow to physical servers.

Deploy to Bare Metal

```
# 1. Create a bare-metal workload spec with annotations
$ aether template ml-training --name gpu-cluster-node

# 2. Add Metal3 annotations
#   aether.io/boot-mac-address, aether.io/image-url, etc.

# 3. Deploy to Metal3
$ aether run workload.yaml
Creating BareMetalHost: gpu-cluster-node ... done
Creating userdata Secret: gpu-cluster-node-userdata ... done
Creating networkdata Secret: gpu-cluster-node-networkdata ... done
Waiting for provisioning ... inspecting ... provisioning ... provisioned (8m 32s)

Workload gpu-cluster-node deployed to metal3

# 4. Check provisioning status
$ aether status gpu-cluster-node
```

NAME	RUNTIME	STATE	READY	AGE
gpu-cluster-node	metal3	Provisioned	true	9m

BMC

Power Management

UEFI

Default Boot Mode

8

Annotations Supported

PXE

Network Boot

Bare Metal, Kubernetes Native

Aether's Metal3 adapter turns physical servers into declarative resources. Write a spec, add annotations for your hardware, and let Metal3 handle the provisioning lifecycle.